INSTITUT
GRASSET

PROGRAMMATION AVANCÉE DE BASE DE DONNÉES

TP#1

Réservations de lieux — Seigneur des Anneaux

Format : Application Windows Forms, SQL FIRST, ADO.NET (connecté + déconnecté), Repository Pattern

Équipe : 2 personnes

Pondération : 10 %

Objectifs

1. Concevoir un **schéma relationnel** cohérent et le déployer en **SQL Server** (SQL FIRST).
2. Maîtriser **ADO.NET mode connecté** (lecture séquentielle, requêtes paramétrées, sécurité).
3. Maîtriser **ADO.NET mode déconnecté** (DataAdapter, DataSet, DataTable, CommandBuilder, Update).
4. Implémenter une **architecture en couches** avec **Repository Pattern** (interface générique + classe de base + repos spécialisés).
5. Développer une **UI WinForms** fonctionnelle (DataGridView + contrôles d'édition et recherche).
6. Implémenter une **validation métier** non triviale : *détection de conflit temporel*.

Instructions de remise

1) Déposer un **.zip** contenant : solution Visual Studio, code complet, DatabaseScripts, README.md, captures.
2) Retirer : .git, .vs, bin, obj, packages inutiles.
3) Le projet doit **s'ouvrir et compiler** sans modification (**connectionString fournie**).
4) **Aucun retard** accepté (note 0 passé la date).
5) Document **équipe.txt** : noms des membres de l'équipe.

Contexte thématique — *Seigneur des Anneaux*

1. Le *Haut Conseil* a besoin d'un outil pour **planifier des missions** à travers la Terre du Milieu.
   Vous devez bâtir une application pour :
   a. gérer les **Lieux** (Minas Tirith, Fondcombe, Erebor, etc.);
   b. gérer les **Personnages** (Aragorn, Gandalf, Eowyn, etc.);
   c. planifier des **Missions** dans des plages de temps, **sans chevauchement** dans un même lieu.

Contraintes techniques (obligatoires)

1. **Windows Forms**.
2. **SQL FIRST** : BD **SQL Server** créée par **scripts SQL** (fichiers .sql) *ou* par **exécution automatique** des scripts au démarrage (via ADO.NET).
3. **ADO.NET uniquement** :
    a. **Connecté** : SqlConnection, SqlCommand, SqlParameter, SqlDataReader.
    b. **Déconnecté** : SqlDataAdapter, DataSet, DataTable, SqlCommandBuilder.
4. **Repository Pattern** :
    a. IRepository<T> générique,
    b. RepositoryBase<T> (gère la connectionString et l'ouverture/fermeture),
    c. LieuRepository, PersonnageRepository, MissionRepository.
5. **ConnectionString fournie** (à placer dans app.config, interdiction de la modifier) :

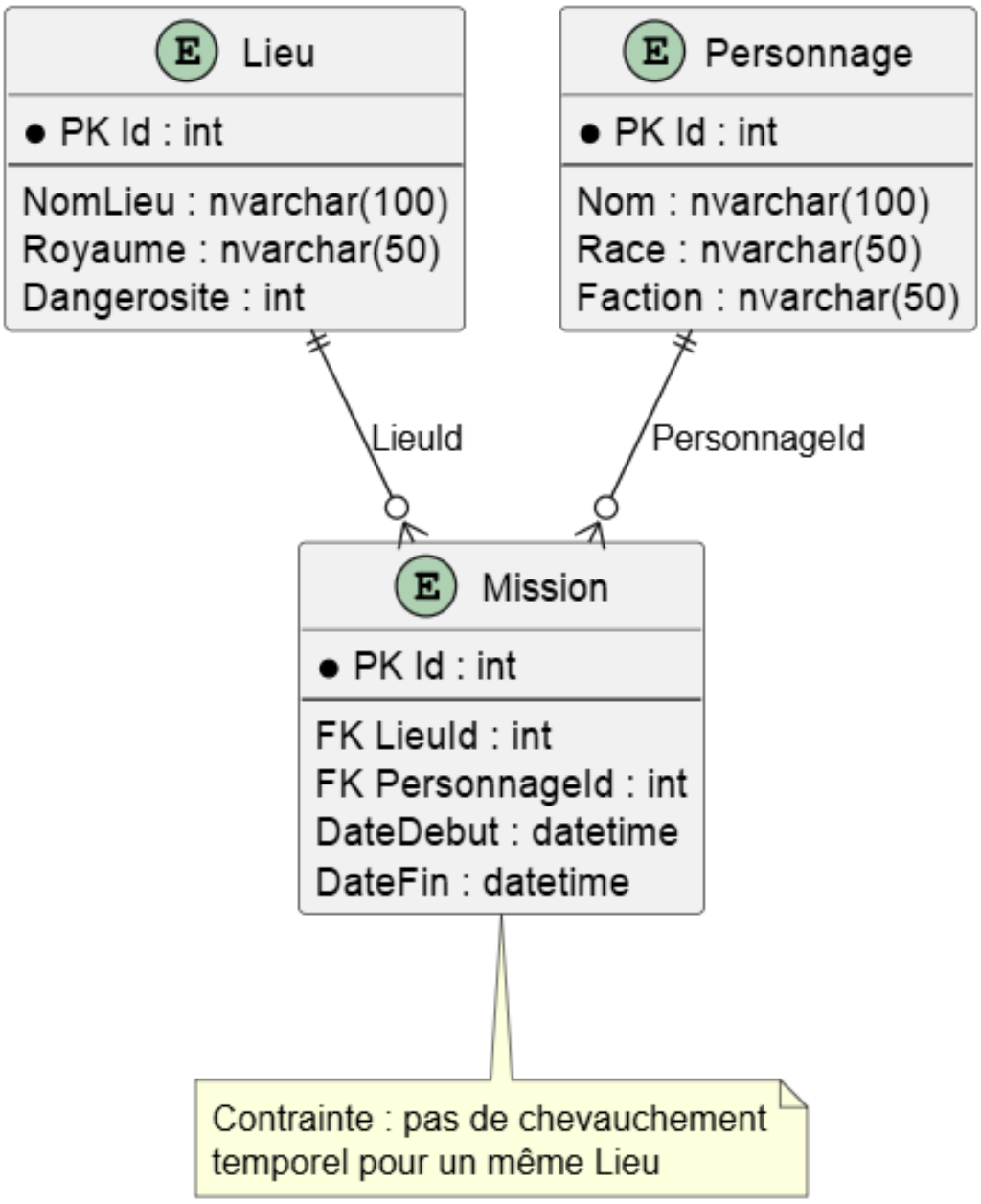
```
"ConnectionStrings":
{
        "DefaultConnection":
        "Server=(localdb)\\mssqllocaldb;Database=TP1_LSDA;Trusted_Connection=True;Multip
        leActiveResultSets=true"
}
```

6. **Aucun EF / Aucune migration**.
7. **Aucun SQL dans l'UI** (tout passe par les repositories).

Travail à faire

1. **Base de données (SQL FIRST)**
    a. Fournir
        i. create_database.sql
        ii. create_tables.sql
        iii. insert_data.sql
    b. Exécuter dans SSMS **ou** déclencher automatiquement au démarrage de l'app via ADO.NET.
2. **Domain & Repositories**
    a. Créer les entités **Lieu**, **Personnage**, **Mission**.
    b. Créer **IRepository<T>, RepositoryBase<T>** et un repository par entité.
    c. Aucune logique SQL dans les formulaires.
3. **UI WinForms (3 onglets)**
    a. **Onglet Lieux** (**CRUD déconnecté** : DataAdapter/DataSet + DataGridView).
    b. **Onglet Personnages** (**CRUD déconnecté**).
    c. **Onglet Missions** :
        i. Formulaire de **création** (combos Lieu/Personnage + DateDebut/DateFin).
        ii. **Vérification de conflit** en **mode connecté** (requête paramétrée + DataReader).
        iii. **Recherche filtrée** en **mode connecté** (ex. missions par période, lieux par royaume, personnages par race).
        iv. **Affichage global** des missions (**déconnecté**) dans un DataGridView (avec jointure ou colonnes calculées pour montrer Lieu/Personnage).
4. **Validation & Erreurs**
    a. Empêcher toute mission dont la plage **chevauche** une mission existante du **même lieu**.
    b. Valider champs vides, formats, bornes de dates (DateFin > DateDebut).
    c. Messages clairs à l'utilisateur (MessageBox).
5. **Documentation**
    a. **README** technique (voir section dédiée).
    b. **Diagramme BD** inséré dans le README.
    c. Captures d'écran de l'application (chaque onglet).

Diagramme de BD



Exigences fonctionnelles

1) **Lieux — CRUD complet (déconnecté)** : créer, lire, modifier, supprimer.
2) **Personnages — CRUD complet (déconnecté)** : idem.
3) **Missions — CRUD partiel** :
   a) **Créer** une mission avec sélection Lieu/Personnage, DateDebut, DateFin.
   b) **Vérifier conflit** en **connecté avant insertion** (requête paramétrée + lecture DataReader).
   c) **Lister** toutes les missions en **déconnecté** (DataGridView alimenté par DataSet).
4) **Recherche filtrée (connecté)** : au moins **une** recherche parmi :
   a) missions par plage de dates,
   b) lieux par royaume,
   c) personnages par race,
   d) lieux « disponibles » sur une plage (option plus avancée).
5) **UX** : boutons **Ajouter / Sauvegarder / Supprimer** sur les onglets Lieux/Personnages, rafraîchissement de la grille après opérations, messages clairs.

Structure du projet

1) /UI
   a) FrmMain.cs
   b) FrmLieux.cs
   c) FrmPersonnages.cs
   d) FrmMissions.cs
2) /Domain
   a) Lieu.cs
   b) Personnage.cs
   c) Mission.cs
3) /Repositories
   a) IRepository.cs
   b) RepositoryBase.cs
   c) LieuRepository.cs
   d) PersonnageRepository.cs
   e) MissionRepository.cs
4) /DatabaseScripts
   a) create_database.sql
   b) create_tables.sql
   c) insert_data.sql
5) app.config
6) Program.cs
7) README.md

Détails obligatoires des onglets WinForms

1) **Nombre d'onglets : 3 (exactement)**
   a) **Onglet 1 — Lieux**
      i) DataGridView dgvLieux (données **déconnectées**)
      ii) Boutons : **Ajouter**, **Sauvegarder**, **Supprimer**
      iii) Zones d'édition (ou colonnes éditables) : NomLieu, Royaume, Dangerosite
   b) **Onglet 2 — Personnages**
      i) DataGridView dgvPersonnages (**déconnecté**)
      ii) Boutons : **Ajouter**, **Sauvegarder**, **Supprimer**
      iii) Zones d'édition : Nom, Race, Faction
   c) **Onglet 3 — Missions**
      i) Section **Création** : cbLieu, cbPersonnage, dtpDebut, dtpFin, bouton de création
      ii) **Vérification de conflit** : requête **connectée** + SqlDataReader **avant** l'insertion
      iii) **Recherche filtrée** (**connectée**) : filtres (période/royaume/race), btnRechercher, dgvResultats
      iv) **Liste globale** (**déconnectée**) : dgvMissions alimenté par DataSet (affiche Lieu/Personnage/DateDebut/DateFin)

README — Modèle de contenu

1) **Titre** : Réservations de Lieux — *Seigneur des Anneaux* (WinForms, SQL FIRST, ADO.NET)
   a) **Description**
   But de l'application, technologies utilisées, structure en couches.
   b) **Architecture**
2) Schéma d'arborescence du projet.
   a) Brève explication du **Repository Pattern** (IRepository, RepositoryBase, repos spécialisés).
   b) Description des **modes ADO.NET** utilisés (connecté pour recherche & conflits, déconnecté pour CRUD/listes).
3) **Base de données**
   a) Insérer le **diagramme BD**.
   b) Lister les tables et contraintes.
   c) Expliquer comment exécuter les scripts (ordre recommandé).
4) **Exécution**
   a) Prérequis (SQL Server/SSMS).
   b) Comment lancer la solution, où est la connectionString, comment initialiser la BD.
   c) Comptes/données d'essai (si pertinentes).
5) **Fonctionnalités**
   a) Ce que fait chaque onglet.
   b) Comment effectuer une recherche, créer une mission, etc.
6) **Tests / Validation**
   a) Cas de conflit attendu (exemple précis).
   b) Cas de saisies invalides (dates inversées, champs vides).
7) **Captures d'écran**
   a) Un screenshot par onglet.
8) **Équipe**
   a) Noms, rôles, répartition des tâches.

Grille de correction

Catégorie	Détails	Points
SQL FIRST & scripts	BD complète (tables, PK, FK), scripts propres et exécutables, données d'essai cohérentes	**20**
ADO.NET — Connecté	Recherche filtrée réelle (SqlParameter), DataReader, sécurité (pas de concaténation), vérif. de conflit **avant** insertion	**20**
ADO.NET — Déconnecté	DataAdapter/DataSet, CRUD via DataGridView, SqlCommandBuilder + Update(), synchronisation fiable	**20**
Repository Pattern	IRepository<T>, RepositoryBase<T>, repos spécialisés, **aucun SQL dans l'UI**, code factorisé	**15**
WinForms — 3 onglets	Respect strict des 3 onglets, libellés clairs, rafraîchissements, combos alimentés, UX correcte	**10**
Validation & Erreurs	Détection chevauchement, validation champs/dates, messages utilisateur clairs, gestion exceptions	**10**
README & Diagramme BD	README technique complet, diagramme BD inséré, instructions d'exécution, captures, contributions	**5**
Total		**100**

Malus

Orthographe et qualité de la langue :

- -1 à -10 points selon la fréquence et la gravité des fautes.

Mauvaise remise

- Projet non remis dans le format demandé (ex. archive mal nommée, fichiers manquants) → -5 points;

Plagiat

- Note finale = 0 (aucune tolérance).

Références

- SQL Server: https://learn.microsoft.com/fr-ca/sql/?view=sql-server-ver17
- ADO .NET: https://learn.microsoft.com/fr-ca/dotnet/framework/data/adonet/
- GIT: https://git-scm.com/doc

Politique sur l'utilisation de l'IA

L'utilisation d'outils d'intelligence artificielle (IA) tels que *Copilot*, *ChatGPT* ou autres est strictement limitée dans le cadre de ce travail.

- Les étudiants peuvent s'en servir comme outil d'aide (par exemple pour chercher de la documentation, obtenir des explications ou générer des idées).
- En revanche, la production directe de contenu par l'IA (code, documentation, rapports, schémas, etc.) et remise telle quelle sera considérée comme du plagiat.
- Tout travail dont le style, la structure ou le contenu laisse paraître une utilisation manifeste de l'IA avec appropriation personnelle sera sanctionné par une note finale de 0.